RAPPORT TECHNIQUE

Diagnostic de Défauts dans les Moteurs CC 12V

Classification Binaire par Apprentissage Automatique
pour Déploiement sur Microcontrôleur ESP32

Projet	Système embarqué de détection de défauts — Moteur CC 12V
Plateforme	ESP32 (microcontrôleur basse consommation)
Approche	Classification supervisée binaire à partir de signaux capteurs
Dataset	ITSC Dataset — Signaux de courant triphasé d'un moteur à induction
Évaluation	Leave-One-Out (LOO) + SMOTE dans Pipeline scikit-learn
Classes	Normal (0) / Surcharge (1)

Note méthodologique — v2.0 : La version précédente utilisait un split 80/20 produisant ~1 échantillon Normal dans le jeu de test — résultats statistiquement non significatifs. Ce rapport adopte Leave-One-Out + SMOTE dans un Pipeline scikit-learn propre pour une évaluation rigoureuse.

1. Introduction

1.1 Objectif du projet

Ce projet vise à développer un système de diagnostic automatique pour les petits moteurs à courant continu (CC) 12V. Le système doit être capable de classifier en temps réel l'état de fonctionnement du moteur en **deux catégories** :

- **Normal** : le moteur fonctionne dans ses conditions nominales
- **Surcharge** : une contrainte mécanique ou électrique excessive est détectée

Le diagnostic repose sur l'analyse de signaux issus de trois capteurs :

- Capteur de courant (ex. ACS712) — mesure du courant consommé
- Capteur de tension (diviseur de tension) — mesure de la tension d'alimentation
- Capteur de vitesse (encodeur ou capteur à effet Hall) — mesure de la vitesse de rotation

1.2 Contrainte matérielle

Le modèle de classification entraîné est destiné à être déployé sur un microcontrôleur **ESP32**, ce qui impose des contraintes strictes :

Contrainte	Implication
Mémoire RAM limitée (~520 Ko)	Le modèle doit avoir une empreinte mémoire de quelques Ko
Pas de GPU	Les calculs doivent se faire en CPU avec arithmétique flottante simple
Pas de bibliothèques ML	Le modèle doit être exporté en code C natif
Temps réel	L'inférence doit se faire en moins de 100 ms

Ces contraintes excluent les architectures complexes (réseaux de neurones profonds, CNN, LSTM) et orientent le choix vers des algorithmes classiques de machine learning.

2. Analyse du Dataset

2.1 Présentation du dataset ITSC

Le dataset utilisé est le ITSC Dataset (Inter-Turn Short-Circuit), développé par le Laboratoire de Génie Électrique de l'Université de Guanajuato (Mexique). Il est disponible publiquement sur GitHub : https://github.com/ibarram/ITSC

Ce dataset contient des signaux de courant triphasé mesurés sur un moteur à induction à cage d'écureuil (Baldor CM3542, 0.75 HP, 230 VAC, 1725 RPM). Le moteur a été instrumenté avec des commutateurs permettant de simuler des défauts de court-circuit inter-spires à différents niveaux de sévérité.

Référence scientifique : Cardenas-Cornejo, J.-J. et al., "Classification of inter-turn short-circuit faults in induction motors based on quaternion analysis", Measurement, Volume 222, 2023.

2.2 Structure du dataset

Le dataset est fourni en trois versions :

Dossier	Contenu	Dimensions par fichier	Total
RAW_Signals	Signaux bruts non traités, tous dans un seul dossier	5000 × 3	65 fichiers
RAW_Signals_SF	Mêmes signaux bruts, organisés par sous-dossiers de classe	5000 × 3	13 dossiers × 5 fichiers
Cropped_Signals_SF	Signaux pré-traités et recadrés, par sous-dossiers	1000 × 3	13 dossiers × 5 fichiers

Pour ce projet, nous utilisons Cropped_Signals_SF car :

- Les signaux sont déjà filtrés et centrés sur l'état stationnaire du défaut
- La dimension réduite (1000 au lieu de 5000 échantillons) est plus adaptée au traitement ESP32
- L'organisation par sous-dossiers facilite le chargement et l'attribution des classes

2.3 Description des classes originales

Chaque fichier CSV contient une matrice de **1000 lignes × 3 colonnes**, où :

- Les 1000 lignes représentent des points temporels (1 seconde à 1 kHz)
- Les 3 colonnes correspondent aux 3 phases de courant (A, B, C) du moteur

Le dataset comporte **13 classes**, soit 65 fichiers au total :

Classe	Description	Nb fichiers
SC_HLT	État sain (Healthy) — aucun défaut	5
SC_A1_B0_C0	Court-circuit phase A à 10 %	5
SC_A2_B0_C0	Court-circuit phase A à 20 %	5
SC_A3_B0_C0	Court-circuit phase A à 30 %	5
SC_A4_B0_C0	Court-circuit phase A à 40 %	5

Classe	Description	Nb fichiers
SC_A0_B1_C0	Court-circuit phase B à 10 %	5
SC_A0_B2_C0	Court-circuit phase B à 20 %	5
SC_A0_B3_C0	Court-circuit phase B à 30 %	5
SC_A0_B4_C0	Court-circuit phase B à 40 %	5
SC_A0_B0_C1	Court-circuit phase C à 10 %	5
SC_A0_B0_C2	Court-circuit phase C à 20 %	5
SC_A0_B0_C3	Court-circuit phase C à 30 %	5
SC_A0_B0_C4	Court-circuit phase C à 40 %	5

Convention de nommage : SC_AX_BX_CX_00Y où X est le niveau de défaut (0–4) pour chaque phase et Y le numéro de répétition (1–5).

2.4 Acquisition des données

Les mesures ont été réalisées avec :

- Capteurs : Transformateurs de courant split-core SCT013
- Module d'acquisition : NI-9215 Series
- Logiciel : LabVIEW
- Conditions : État stationnaire, sans charge, à 60 Hz
- Fréquence d'échantillonnage : 1 kHz
- Durée d'acquisition : 5 secondes (signaux bruts), recadrés à 1 seconde

2.5 Adaptation au problème à 2 classes

Le dataset original comporte 13 classes, mais notre problème nécessite seulement **2 classes**. Le remapping suivant a été défini :

Classe cible	Classes ITSC sources	Nb fichiers	Justification
Normal	SC_HLT	5	État sain, fonctionnement nominal
Surcharge	Niveaux 1–2 (10 %–20 %) des 3 phases	30	Les défauts légers produisent des signatures similaires à une surcharge mécanique
(Exclus)	Niveaux 3–4 (30 %–40 %) des 3 phases	30	Court-circuit franc — hors périmètre du classificateur binaire

Justification du regroupement :

- Les défauts inter-spires à faible niveau (10–20 %) se manifestent par une légère augmentation du courant et une distorsion mineure du signal, comportement analogue à une surcharge
- Les défauts à niveau élevé (30–40 %) provoquent des perturbations majeures — traités dans un notebook de classification ternaire séparé
- Ce remapping produit un dataset de **35 échantillons** (5 Normal + 30 Surcharge) avec un ratio de déséquilibre de 1:6

2.6 Correspondance des colonnes aux capteurs

Les 3 colonnes du dataset (phases A, B, C) sont mappées aux 3 capteurs du système embarqué :

Colonne CSV	Phase ITSC	Capteur ESP32
Colonne 0	Phase A	Courant
Colonne 1	Phase B	Tension
Colonne 2	Phase C	Vitesse

Ce mapping est cohérent car le modèle apprend les relations entre 3 signaux simultanés, quelle que soit leur nature physique exacte.

3. Extraction des caractéristiques (Feature Engineering)

3.1 Pourquoi extraire des caractéristiques ?

Les signaux bruts (1000 × 3 = 3000 valeurs par échantillon) ne peuvent pas être utilisés directement comme entrée du modèle sur ESP32, car :

1. La dimension est trop élevée pour une inférence rapide
2. Les algorithmes classiques (arbres, SVM) ne traitent pas efficacement les séries temporelles
3. Les caractéristiques statistiques capturent l'essence du signal en un vecteur compact

3.2 Caractéristiques extraites

Pour chaque capteur (3 au total), on extrait **15 caractéristiques**, soit **45 au total**.

Caractéristiques temporelles (10 par capteur)

#	Caractéristique	Formule	Signification physique
1	Moyenne	$\mu = (1/N)\ \Sigma xi$	Composante continue du signal
2	Écart-type	$\sigma = \sqrt{[(1/N)\ \Sigma(xi-\mu)^2]}$	Dispersion autour de la moyenne
3	Valeur RMS	$\sqrt{[(1/N)\ \Sigma xi^2]}$	Valeur efficace (puissance du signal)
4	Maximum	max(x)	Pic maximal du signal
5	Minimum	min(x)	Creux minimal du signal
6	Amplitude crête-à-crête	max(x) − min(x)	Étendue totale du signal
7	Kurtosis	$E[(x-\mu)^4]/\sigma^4 - 3$	Présence de pics ou de valeurs extrêmes
8	Asymétrie (Skewness)	$E[(x-\mu)^3]/\sigma^3$	Asymétrie de la distribution
9	MAD	$(1/N)\ \Sigma\lvert xi - xi{-}1\rvert$	Rugosité/variabilité du signal
10	IQR	Q75 − Q25	Dispersion des valeurs centrales

Caractéristiques fréquentielles (5 par capteur)

#	Caractéristique	Description
11	Énergie spectrale	Somme des carrés du spectre FFT
12	Fréquence dominante	Fréquence ayant la plus grande amplitude
13	Fréquence moyenne pondérée	Barycentre fréquentiel
14	Dispersion spectrale	Écart-type des amplitudes spectrales
15	Ratio du pic spectral	Rapport entre le pic maximal et la somme totale

3.3 Pourquoi ces caractéristiques ?

- **Caractéristiques temporelles** : elles capturent les modifications d'amplitude, de forme et de distribution du signal causées par les défauts. Une surcharge augmente le RMS et le peak-to-peak, et modifie la kurtosis ainsi que l'asymétrie.
- **Caractéristiques fréquentielles** : les défauts introduisent des harmoniques supplémentaires dans le spectre. La fréquence dominante et l'énergie spectrale sont des indicateurs fiables de l'état du moteur.

- **Calculables sur ESP32** : toutes ces caractéristiques ne nécessitent que des opérations arithmétiques simples (addition, multiplication, comparaison) facilement implémentables en C.

4. Préparation des données

4.1 Protocole d'évaluation : Leave-One-Out (LOO)

Avec seulement 35 échantillons disponibles, un split 80/20 classique produirait un jeu de test de ~7 points dont **un seul échantillon Normal** — insuffisant pour des métriques statistiquement significatives. On adopte la méthode **Leave-One-Out (LOO)** :

- À chaque itération, un seul échantillon sert de jeu de test (1 point)
- Les 34 échantillons restants constituent le jeu d'entraînement
- Le processus se répète 35 fois — chaque point est évalué exactement une fois
- Les métriques sont calculées sur l'ensemble des 35 prédictions indépendantes

> Le LOO est la méthode de référence pour les petits datasets : il utilise au maximum les données disponibles et produit une estimation quasi sans biais de la capacité de généralisation.

4.2 Normalisation

Les caractéristiques sont normalisées avec le **StandardScaler** de scikit-learn, qui transforme chaque caractéristique pour avoir une moyenne de 0 et un écart-type de 1 :

```
x_normalisé = (x - moyenne) / écart-type
```

La normalisation est indispensable car :

- Les caractéristiques ont des échelles très différentes (l'énergie spectrale est de l'ordre de 10^5, la fréquence dominante est de l'ordre de 10^1)
- Les algorithmes comme KNN, SVM et MLP sont sensibles à l'échelle des données
- Le StandardScaler est refitté à chaque fold LOO uniquement sur les données d'entraînement, puis appliqué au point de test
- Les paramètres du scaler sont exportés dans **scaler_params_binary.h** pour être reproduits sur ESP32

4.3 Gestion du déséquilibre de classes (SMOTE)

Le dataset est fortement déséquilibré : 5 Normal contre 30 Surcharge (ratio 1:6). La technique **SMOTE (Synthetic Minority Over-sampling Technique)** génère des échantillons synthétiques de la classe minoritaire par interpolation entre voisins réels.

Mécanisme	Principe	Appliqué à
SMOTE (k=3)	Génère des échantillons synthétiques Normal par interpolation. k=3 car le fold LOO ne contient que 4 Normal.	À l'intérieur de chaque fold LOO via ImbPipeline
class_weight='balanced'	Pondère les erreurs inversement à la fréquence de classe.	Decision Tree, Random Forest, SVM, Reg. Logistique, HistGBT
scale_pos_weight=6	Équivalent de class_weight pour XGBoost (ratio 30/5).	XGBoost uniquement

> **Règle fondamentale :** SMOTE est appliqué *à l'intérieur de chaque fold LOO via ImbPipeline*, jamais avant. Cela garantit que le point de test est toujours un vrai échantillon réel — aucune fuite de données synthétiques.

5. Modèles de classification

5.1 Choix des algorithmes

Huit algorithmes ont été évalués, choisis pour leur compatibilité avec les contraintes ESP32 :

Algorithme	Principe	Avantage ESP32	Limitation
Decision Tree	Séquence de tests sur les caractéristiques	Très compact (quelques Ko), rapide	Peut sur-apprendre
Random Forest	Ensemble de 10 arbres de décision	Plus robuste qu'un seul arbre	Taille proportionnelle au nb d'arbres
KNN	Comparaison aux k voisins les plus proches	Pas d'entraînement	Nécessite de stocker toutes les données
SVM linéaire	Hyperplan séparateur de marge maximale	Très compact (poids + biais)	Limité aux frontières linéaires
Régression logistique	Modèle linéaire probabiliste	Le plus compact possible	Limité aux frontières linéaires
Perceptron multicouche	Réseau de neurones à 1–2 couches cachées	Taille contrôlable	Plus lent à inférer
HistGradientBoosting	Boosting avec histogrammes	Très performant, relativement compact	Plus complexe qu'un arbre unique
XGBoost	Extreme Gradient Boosting	État de l'art pour données tabulaires	Taille du modèle variable

5.2 Optimisation des hyperparamètres

Les hyperparamètres sont **pré-sélectionnés** sur la base des résultats de la version multi-classe. Un GridSearchCV imbriqué dans le LOO serait trop coûteux sur 35 échantillons.

Les valeurs fixées pour les paramètres clés sont :

- Decision Tree / Random Forest : max_depth=5, min_samples_split=2, class_weight='balanced'
- SVM linéaire : C=1.0, dual=False, class_weight='balanced'
- Régression Logistique : C=1.0, max_iter=2000, class_weight='balanced'
- Perceptron Multicouche : hidden_layer_sizes=(20,), activation='relu'
- HistGradientBoosting : max_iter=100, max_depth=5, class_weight='balanced'
- XGBoost : n_estimators=50, max_depth=3, learning_rate=0.1, scale_pos_weight=6

5.3 Métriques d'évaluation

Les modèles sont évalués selon :

- **Balanced Accuracy (Accuracy Équilibrée)** : moyenne du recall de chaque classe — insensible au déséquilibre 1:6. Métrique principale. Un modèle qui prédit toujours "Surcharge" obtient 0.50 (ligne de base aléatoire).
- **F1-Score (classe Surcharge)** : moyenne harmonique de la précision et du rappel pour la classe Surcharge.
- **Accuracy par classe** : Acc. Normal (sur 5 pts) et Acc. Surcharge (sur 30 pts) pour détecter les biais par classe.
- **Matrice de confusion** : visualisation des erreurs par classe sur les 35 prédictions LOO.

6. Résultats de l'Évaluation LOO + SMOTE

6.1 Tableau récapitulatif

Les métriques ci-dessous sont calculées sur les **35 prédictions LOO** (chaque échantillon réel prédit exactement une fois). Le tableau est trié par Accuracy Équilibrée décroissante :

Modèle	Acc. Éq. (principale)	F1 (Surcharge)	Accuracy	Acc. Normal	Acc. Surcharge	Temps (s)
Régression Logistique	**0.967**	**0.966**	**0.943**	**1.000**	**0.933**	**0.097**
KNN	0.900	0.889	0.829	1.000	0.800	0.083
SVM Linéaire	0.883	0.967	0.943	0.800	0.967	0.081
Random Forest	0.883	0.967	0.943	0.800	0.967	0.186
Decision Tree	0.867	0.949	0.914	0.800	0.933	7.532
HistGradientBoosting	0.783	0.951	0.914	0.600	0.967	0.418
XGBoost	0.783	0.951	0.914	0.600	0.967	1.110
Perceptron Multicouche	0.583	0.524	0.429	0.800	0.367	0.203

Légende : **Vert** = *meilleur modèle ESP32-compatible* | **Orange** = *performance insuffisante* | *KNN = non compatible ESP32 (stockage des données d'entraînement requis)*

6.2 Analyse des résultats

La **Régression Logistique** obtient la meilleure Accuracy Équilibrée (0.967) : elle classe correctement les 5 échantillons Normal (Acc. Normal = 1.0) et 28/30 échantillons Surcharge (Acc. Surcharge = 0.933). C'est également le modèle le plus rapide parmi les compatibles ESP32 (0.097 s).

Le **SVM Linéaire** et le **Random Forest** atteignent une Balanced Accuracy identique (0.883) avec un F1 très élevé (0.967), mais au prix d'un Normal mal classifié (Acc. Normal = 0.800).

Le **Perceptron Multicouche** est le seul modèle clairement insuffisant (Balanced Accuracy = 0.583), proche de la ligne de base aléatoire (0.50). Ce résultat s'explique par la sensibilité des réseaux de neurones à la taille du dataset et à l'initialisation des poids.

6.3 Sélection du modèle pour ESP32

Le meilleur modèle compatible ESP32 est la **Régression Logistique** (KNN exclu car il nécessite le stockage de l'ensemble d'entraînement en mémoire) :

Critère	Valeur
Modèle sélectionné	**Régression Logistique**
Balanced Accuracy (LOO)	0.967
F1-score (Surcharge)	0.966
Accuracy globale	0.943

Critère	Valeur
Accuracy Normal (5 pts)	1.000 — 5/5 corrects
Accuracy Surcharge (30 pts)	0.933 — 28/30 corrects
Temps (LOO)	0.097 s
Compatibilité ESP32	Oui — modèle linéaire, très compact

7. Exportation pour ESP32

7.1 Conversion en code C

Le meilleur modèle compatible ESP32 est converti en code C natif à l'aide de la bibliothèque **m2cgen** (Model to Code Generator). Cette bibliothèque traduit les modèles scikit-learn en une fonction C pure `score()` qui :

- Ne dépend d'aucune bibliothèque externe
- Est directement compilable avec le toolchain Arduino/ESP-IDF
- Accepte un tableau de flottants (les 45 caractéristiques normalisées) en entrée
- Renvoie un tableau de **2 scores** (un par classe) dont l'argmax donne la classe prédite

7.2 Fichiers générés

Fichier	Contenu
model_esp32_binary_loo.c	Code C du modèle de classification (fonction score())
scaler_params_binary_loo.h	Tableaux scaler_mean[45] et scaler_scale[45] pour la normalisation

7.3 Intégration sur ESP32

L'intégration sur ESP32 suit le pipeline :

4. Acquisition des 3 signaux (1000 échantillons à 1 kHz)
5. Calcul des 45 caractéristiques en C (même logique que la fonction Python extract_features)
6. Normalisation avec les paramètres de scaler_params_binary_loo.h
7. Appel de la fonction score() de model_esp32_binary_loo.c
8. Sélection de la classe par argmax du tableau de sortie (0=Normal, 1=Surcharge)
9. Déclenchement de l'alarme si classe == 1 (Surcharge détectée)

8. Conclusion

Ce travail a permis de :

10. Analyser et restructurer le dataset ITSC pour un problème de classification binaire (Normal vs Surcharge), en excluant les niveaux de défaut sévères (30–40 %)
11. Identifier et corriger le problème méthodologique du split 80/20 sur un dataset de 35 échantillons — remplacement par Leave-One-Out + SMOTE dans un Pipeline scikit-learn propre
12. Extraire 45 caractéristiques pertinentes (temporelles et fréquentielles) de chaque signal
13. Entraîner et comparer 8 algorithmes de classification avec évaluation LOO rigoureuse et gestion explicite du déséquilibre de classes 1:6
14. Identifier la **Régression Logistique** comme meilleur modèle ESP32-compatible (Balanced Accuracy = 0.967, F1 = 0.966)
15. Exporter le modèle en code C directement exécutable sur ESP32 via m2cgen

Le système proposé permet une **détection de surcharge en temps réel** avec une empreinte mémoire minimale, adapté aux microcontrôleurs basse consommation. La méthode d'évaluation LOO garantit que les métriques rapportées reflètent fidèlement la capacité de généralisation du modèle malgré la faible taille du dataset.